

Secure Smart Laboratory Equipment Management System

Harinilekha R – CH.SC.U4CYS25011
Nandika Yazhini – CH.SC.U4CYS25021

Project team

Harinilekha R – CH.SC.U4CYS25011

Nandika Yazhini – CH.SC.U4CYS25021

Certificate

Template only: complete and obtain authorised faculty/department approval.

Declaration

Template only: complete and obtain approval as required by the institution.

Abstract

This project proposes a Raspberry Pi based system for controlled laboratory equipment sharing. RFID UID verification is used to authenticate requests. Equipment availability and mutex-protected allocation are intended to prevent conflicting allocations. SQLite stores records, and FCFS queue management handles requests when equipment is occupied. A servo lock, LCD, LEDs, and buzzer provide intended physical interaction. The supplied presentation specifies AES-encrypted audit logging; implementation and operation must be confirmed against code and test evidence.

Introduction

Laboratories share limited instruments, and simultaneous requests can cause allocation conflicts or poor usage visibility. This academic prototype combines RFID identification, resource allocation, database records, and a queue workflow.

Problem Statement

Provide a consistent process to identify users, check equipment availability, record allocations/returns, and handle requests for occupied equipment fairly.

Objectives

- Authenticate users through RFID UID checking.
- Track equipment availability.
- Prevent conflicting allocations using mutual exclusion.

- Queue requests in FCFS order.
- Record requests and equipment state in SQLite.
- Provide intended status/denied-access feedback.
- Demonstrate OS resource-management concepts.

System Requirements

Hardware: Raspberry Pi 4, RC522 reader/cards, servo lock, 16×2 I2C LCD, LEDs, buzzer, wiring, and suitable power. Software: Raspberry Pi OS/Linux, Python, SQLite, and the required hardware libraries. Confirm exact versions against the actual build.

System Architecture

The PPT describes User, Processing, Data, and Action layers. The user scans an RFID card; the Pi validates the UID; SQLite stores records; the action layer controls the intended servo/LCD/LED/buzzer feedback.

Methodology

1. Read RFID UID.
2. Validate against authorised records.
3. Reject unauthorised requests and issue configured alerts.
4. Check equipment availability.
5. Protect shared allocation/database state with a mutex.
6. Allocate available equipment or enqueue the request in FCFS order.
7. On return, update the database and consider the next waiting request.

Operating System Concepts

Process/request management, scheduling, mutual exclusion, resource allocation, synchronization, and FCFS queue scheduling are demonstrated at application level. The project does not modify Linux kernel scheduling.

Database Design

Logical entities include Students, Equipment, Waiting Queue, and Transactions/Audit Logs. Confirm exact table/column names against `database/schema.sql`.

Security Design

RFID UID checking alone should not be described as strong cryptographic authentication. The PPT specifies AES-encrypted audit logs; verify encryption code, key handling, and tests before claiming implementation is complete.

Demo Scenarios

Successful Access: Student A scans RFID, authentication succeeds, equipment is available, allocation is recorded, and the intended access feedback is triggered.

Conflict Handling: Student B requests occupied equipment, joins FCFS, and is considered after Student A returns it.

Security Breach: An unknown UID is rejected; the design calls for red LED/buzzer feedback and a denied-attempt log.

Testing

Use `testing/test\_cases.md`. Report software test output separately from physical hardware validation. Hardware results remain pending until verified.

Advantages

Structured allocation, fair waiting order, access control, database records, and a practical application of OS concepts.

Limitations

Physical operation depends on wiring, power, drivers, and Raspberry Pi configuration. UID-only checks have security limitations. Encryption depends on correct implementation and key management. Further reliability and security testing is required before production use.

Future Enhancements

Web dashboard, role-based administration, usage analytics, notifications, multiple lockers, and improved key management.

Conclusion

The project proposes integration of RFID access, SQLite records, mutex-protected allocation, FCFS waiting, and physical feedback for laboratory equipment sharing. The final submission must distinguish tested software behaviour from hardware functions awaiting validation.

References

- Team-supplied project presentation, *Secure Smart Laboratory Equipment Management System*.
- Python documentation: <https://docs.python.org/3/>
- SQLite documentation: <https://www.sqlite.org/docs.html>
- Raspberry Pi documentation: <https://www.raspberrypi.com/documentation/>